

Recomendación local de citas académicas mediante aprendizaje automático

Reporte final – Entrega 3

Maestría en Inteligencia Artificial – Universidad de los Andes

Carlos Duque · Carlos Caicedo · José Zambrana · John Arias

Proyecto: Desarrollo de Soluciones

22 de septiembre de 2026

1. Problema, datos y alcance

Encontrar una referencia adecuada para respaldar una afirmación académica puede convertirse en una tarea costosa cuando el autor debe buscar entre miles de publicaciones relacionadas con un mismo tema. La recomendación local de citas busca reducir ese esfuerzo utilizando como punto de partida el fragmento de texto en el que se necesita incorporar una referencia.

Este problema ha sido abordado mediante modelos que utilizan el contexto local de la cita para recuperar y ordenar artículos relevantes. En particular, Gu, Gao y Hahnloser (2022) proponen una arquitectura de recomendación local de citas que combina recuperación y reordenamiento de candidatos.

Pregunta de negocio: ¿Cómo apoyar a un autor de textos académicos en la identificación de artículos potencialmente relevantes para citar utilizando el contexto textual local?

El prototipo desarrollado recibe un fragmento académico en inglés y devuelve una lista ordenada de artículos candidatos. Para cada resultado se presentan el identificador del documento, el título, el resumen y las puntuaciones utilizadas por el recomendador.

El proyecto trabaja con el conjunto `custom` (ACL-200), proveniente del repositorio *Local-Citation-Recommendation* publicado por Gu (2022). El corpus utilizado contiene 63.768 contextos de cita y 19.776 artículos candidatos. Las particiones empleadas incluyen 30.390 consultas de entrenamiento, 9.381 de validación y 9.585 de prueba.

Los archivos de datos se administran con DVC a través de `data/raw.dvc` y `data/processed.dvc`. De esta forma, Git se mantiene enfocado en el código, la configuración y la documentación, mientras DVC conserva el versionamiento de los conjuntos utilizados durante los experimentos.

1.1 Cambios respecto a la segunda entrega

- **Segunda versión del reordenador.** El vector de características pasó de cinco a diez señales, incorporando metadatos derivados del identificador del ACL Anthology. El Recall@10 subió de 0,2882 a 0,3793 y el MRR@10 de 0,1490 a 0,2166 sobre las mismas 9.381 consultas de validación.
- **Empaquetamiento del modelo.** El modelo seleccionado se publicó como el `wheel modelo-citas 0.2.0` en un bucket S3 público. La instalación ya no requiere entrenar localmente ni disponer de credenciales de AWS.
- **Del proceso único a los contenedores.** En la Entrega 2 el tablero ya producía recomendaciones reales, pero servía la línea base TF-IDF desde un solo proceso local lanzado con `uv run tablero`. Ahora la API y el tablero son servicios separados, el modelo que se sirve es el reordenador empaquetado, y ambos se orquestan con Docker Compose detrás de Nginx.
- **Separación de responsabilidades.** El código de experimentación en `src/` se separó del paquete distribuible en `model-package/`, de modo que la imagen de producción no arrastra dependencias de investigación.
- **Extensión de clasificación de función de cita.** Se añadió una línea de trabajo que clasifica la función retórica de cada cita en nueve categorías, con validación humana por dos anotadores y acuerdo medido.
- **Remoto DVC público.** El remoto por omisión pasó a ser una URL HTTPS de solo lectura, para que la recuperación de los datos no dependa de credenciales del equipo.

2. Arquitectura y modelo

La solución se organizó en dos etapas. Primero, TF-IDF recupera los artículos más parecidos al contexto de consulta. Después, un modelo supervisado vuelve a ordenar esos candidatos para intentar ubicar la cita correcta entre las primeras posiciones.

Contexto → TF-IDF → Top-100 → reordenador lineal → Top-K → `wheel modelo-citas` → FastAPI → Nginx/tablero

La primera etapa compara el contexto con los 19.776 artículos del corpus mediante similitud coseno y conserva los 100 candidatos con mayor puntaje.

Sobre esos candidatos se aplica una regresión logística que utiliza cinco características:

- similitud TF-IDF entre el contexto y el título;
- similitud TF-IDF entre el contexto y el resumen;
- longitud del contexto;
- longitud del título;
- longitud del resumen.

Para entrenar el reordenador se construyeron aproximadamente 91.170 pares contexto-artículo. Cada consulta aporta una cita positiva y dos ejemplos negativos.

Se probaron dos formas de seleccionar esos negativos. En la primera, **random**, los artículos incorrectos se escogen aleatoriamente. En la segunda, **hard**, se utilizan artículos incorrectos que TF-IDF considera muy parecidos al contexto.

Las siguientes capturas corresponden a las ejecuciones utilizadas para entrenar ambas variantes.

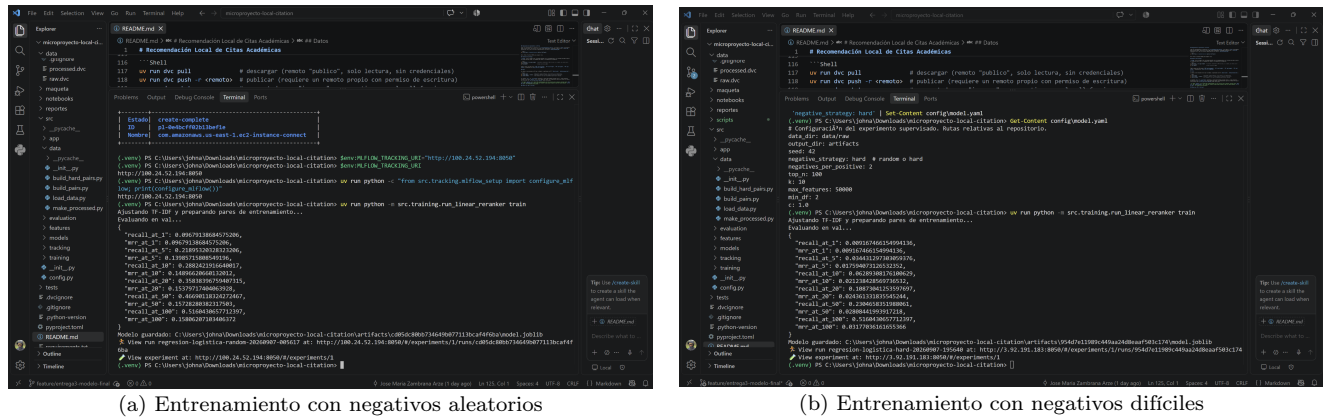


Figura 1: Evidencias de entrenamiento de las dos estrategias evaluadas.

3. Resultados experimentales

Las dos variantes se evaluaron con las mismas particiones de datos, la misma semilla y los mismos parámetros generales. La única diferencia controlada fue la estrategia utilizada para seleccionar los ejemplos negativos.

Los resultados principales fueron los siguientes:

Tabla 1: Comparación de los modelos sobre las 9.381 consultas de validación

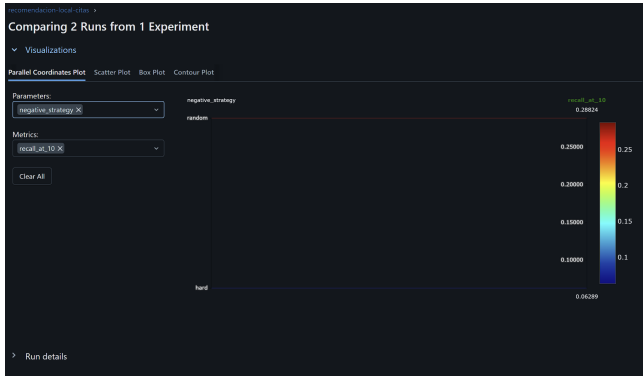
Modelo	Recall@10	MRR@10	Recall@100
TF-IDF sin reordenar	0,2571	0,1339	0,5160
v1 · Random	0,2882	0,1490	0,5160
v1 · Hard	0,0629	0,0212	0,5160
v2 · Random	0,3793	0,2166	0,5160
v2 · Hard	0,2288	0,0995	0,5160

El *Recall@100* se mantiene en 0,5160 en las cinco filas. Esa invariancia comprueba que el reordenador opera efectivamente como segunda etapa –recibe el mismo conjunto recuperado y solo altera el orden– y marca a la vez el techo del sistema: mientras la primera etapa no recupere el artículo correcto, ningún reordenamiento puede corregirlo.

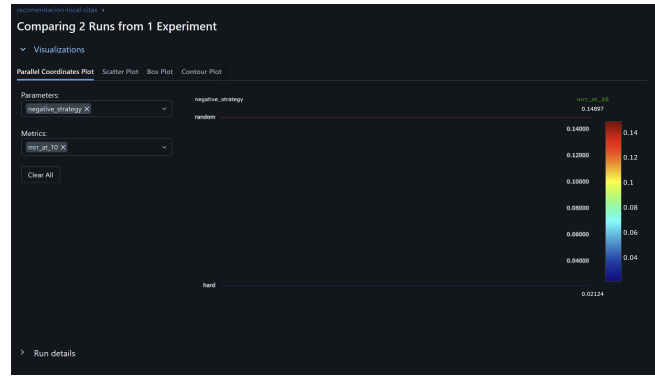
La línea base de comparación es 0,2571 / 0,1339 y no 0,2541 / 0,1249, porque excluye el artículo citante igual que lo hace el conjunto de candidatos que consume el reordenador. Compararlo contra la versión que sí lo incluye inflaría la mejora atribuida al modelo.

En la primera versión (v1), la diferencia entre las dos estrategias es considerable en las primeras posiciones del ranking. Con negativos aleatorios, Recall@10 alcanza 0,2882, mientras que con negativos difíciles baja a 0,0629. El mismo comportamiento aparece en MRR@10, que pasa de 0,1490 a 0,0212.

Las gráficas siguientes permiten observar esta diferencia de manera directa.



(a) Comparación de Recall@10



(b) Comparación de MRR@10

Figura 2: Desempeño de las estrategias Random y Hard en las primeras posiciones del ranking.

3.1 Segunda versión: ampliación de las características

La comparación anterior sugirió que una parte importante del cuello de botella estaba en la representación disponible para el reordenador. Entrenado con negativos difíciles, el modelo asignaba un coeficiente de $-2,09$ a la similitud con el resumen. Ese signo es consistente con la señal observada en este conjunto: el área bajo la curva ROC de positivos frente a negativos difíciles usando solo similitud coseno es $0,4053$, por debajo de $0,5$. Con únicamente cinco características léxicas, distinguir un positivo entre candidatos ya muy parecidos resulta especialmente difícil.

La segunda versión amplía el vector a diez características. Las cinco nuevas se derivan de los metadatos que codifica el identificador del ACL Anthology –tipo de publicación, año y número, formato que cumplen los 19.776 artículos del corpus– y del conjunto de entrenamiento:

- **year_diff**: años entre el artículo citante y el candidato.
- **is_future**: candidato publicado después del citante, es decir una cita imposible.
- **same_venue**: coincidencia en el tipo de publicación.
- **title_overlap**: fracción del título presente en el contexto.
- **citation_prior**: logaritmo de las veces que el candidato se cita en entrenamiento.

La señal temporal resultó especialmente útil. Antes de incorporarla, el $11,1\%$ de los cupos del *Top*–100 correspondía a artículos posteriores al citante. En las 9.381 consultas de validación no se observó ningún positivo posterior al artículo citante; por tanto, en esa partición la señal permite penalizar candidatos temporalmente incompatibles sin perder un positivo observado.

Los coeficientes de la mejor variante experimental muestran el peso de estas señales: sobre características estandarizadas, **is_future** obtiene $-3,0790$ y **citation_prior** $+2,6889$, ambos con magnitud comparable o superior a **similarity_abstract** ($+2,4954$). La regresión asigna así una penalización fuerte a los candidatos posteriores al artículo citante, sin aplicar un filtro temporal determinista.

La **mejor variante experimental es la versión 2 con negativos aleatorios**, de acuerdo con las métricas de ranking: frente a la línea base comparable mejora un $47,5\%$ en Recall@10 y un $61,8\%$ en MRR@10; frente a la primera versión, un $31,6\%$ y un $45,4\%$. La ganancia es mayor en MRR que en Recall, un comportamiento coherente con una segunda etapa que reordena candidatos ya recuperados.

Con el vector ampliado, la variante de negativos difíciles mejora 3,6 veces en Recall@10 (de $0,0629$ a $0,2288$), aunque permanece por debajo de la variante aleatoria. La suma de los coeficientes de similitud continúa siendo negativa, de modo que las características nuevas compensan parte de esa señal. Aprovechar mejor los negativos difíciles probablemente requiera una representación semántica más rica, por ejemplo densa, además de las señales tabulares.

Recall@100 permanece alrededor de $0,516$ en las dos configuraciones porque ambas reciben exactamente los mismos 100 candidatos recuperados por TF–IDF. Esto indica que la diferencia entre los experimentos se encuentra en el ordenamiento y no en la recuperación inicial.

El resultado de la estrategia **hard** también permitió identificar una limitación del modelo. Los negativos difíciles son artículos que ya presentan una gran similitud léxica con el contexto y, utilizando únicamente cinco características, el reordenador dispone de poca información adicional para distinguirlos correctamente.

Por métricas se priorizó la estrategia **random**. Es importante distinguir entre el mejor resultado experimental y la versión actualmente servida: el wheel **modelo-citas 0.2.0** validado en Docker conserva **include_metadata=false**, por lo que el modelo desplegado corresponde a la versión 1 con negativos aleatorios y cinco características. Esa versión obtuvo Recall@10 de $0,2882$ y MRR@10 de $0,1490$. La versión 2 queda documentada como la mejor variante experimental y requiere publicarse en un nuevo wheel para convertirse en la versión desplegada.

4. Seguimiento de experimentos con MLflow

Para llevar un registro común de los experimentos realizados por el equipo, el entrenamiento se integró con MLflow. Cada corrida conserva sus parámetros, métricas de ranking, tiempos de procesamiento y artefactos asociados.

La configuración se centralizó en `src/tracking/mlflow_setup.py` para que todas las ejecuciones siguieran la misma estructura de seguimiento.

Este tipo de organización es coherente con las prácticas de MLOps, en las que se busca mantener trazabilidad sobre datos, experimentos, modelos y configuraciones durante el desarrollo de una solución de aprendizaje automático (Treveil & Dataiku Team, 2021).

Las siguientes capturas muestran las corridas Random y Hard registradas en MLflow y la comparación de sus tiempos de procesamiento.



Figura 3: Seguimiento y comparación de los experimentos realizados.

Las siguientes capturas muestran el detalle de las métricas y los parámetros guardados para cada ejecución.

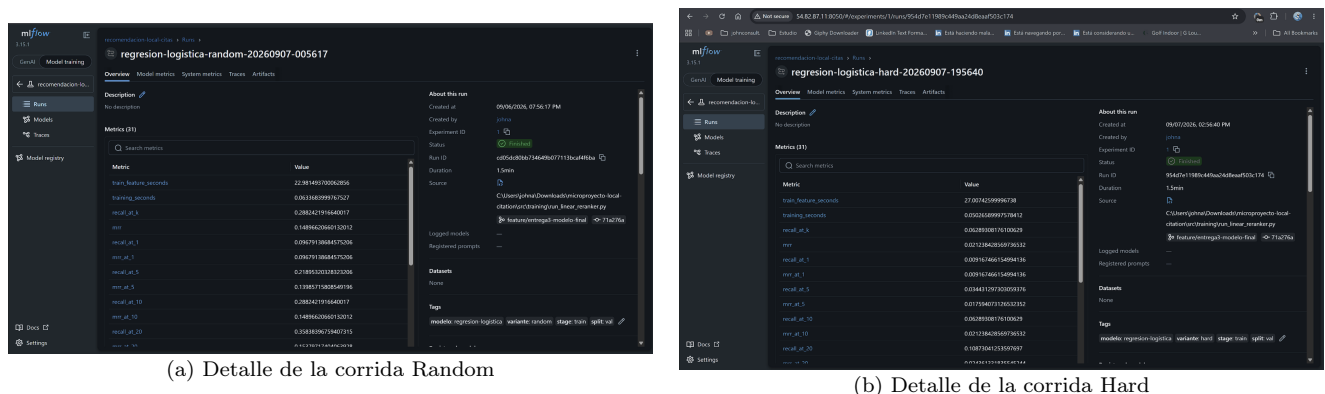


Figura 4: Detalle de las dos variantes de entrenamiento almacenadas en MLflow.

El uso de MLflow permitió comparar las dos variantes sobre una misma base y conservar evidencia de los resultados utilizados para escoger la configuración final del modelo.

5. MLflow en AWS

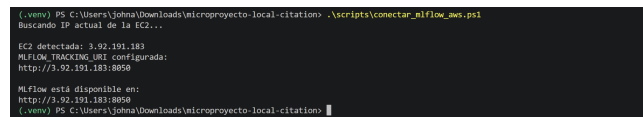
Para que el seguimiento de experimentos no dependiera de una sola máquina local, se instaló MLflow 3.15.1 en una instancia EC2 de AWS Academy con Ubuntu 24.04 LTS.

La instancia utiliza SQLite para guardar los metadatos de MLflow y un directorio local para los artefactos. El servicio escucha por el puerto 8050 y se configuró con `systemd` para que vuelva a iniciarse automáticamente cuando la máquina arranque.

Uno de los inconvenientes encontrados durante el trabajo con AWS Academy fue el cambio de la dirección IP pública después de detener y volver a iniciar la instancia. Para evitar tener que actualizar esta dirección manualmente cada vez, se creó el script `scripts/conectar_mlflow_aws.ps1`.

El script consulta la IP pública actual mediante AWS CLI, configura `MLFLOW_TRACKING_URI` y comprueba que el puerto 8050 sea accesible desde el equipo local.

Las siguientes imágenes muestran el servicio funcionando en EC2 y la conexión automática desde PowerShell.



(b) Detección de la IP y conexión desde el equipo local

(a) Servicio MLflow activo en EC2

Figura 5: Funcionamiento de la infraestructura de MLflow en AWS.

Otro avance de esta entrega fue convertir el recomendador en una librería Python instalable. El código de desarrollo y entrenamiento permanece en `model-package/`, pero la capa de producción consume `modelo-citas` como un wheel versionado. La versión utilizada actualmente es la 0.2.0, publicada en un bucket S3 de solo lectura y fijada por URL y hash en `pyproject.toml` y `uv.lock`.

El wheel incluye el recuperador TF-IDF, el reordenador lineal, la configuración, el modelo entrenado y la información necesaria para generar las recomendaciones. En el artefacto 0.2.0 actualmente publicado, `include_metadata=false`; por ello el runtime sirve la variante v1 **Random** de cinco características. La variante v2 permanece como mejor resultado experimental, pero no se presenta en este reporte como ya desplegada. Por esta separación, FastAPI y las imágenes Docker no necesitan entrenar el modelo, copiar `model-package/` ni descargar el dataset para servir inferencias.

La interfaz pública del paquete puede utilizarse de esta forma:

```
from modelo_citas import make_prediction

resultado = make_prediction(
    context="Recent work on neural machine translation with attention",
    top_k=5,
)
```

La validación posterior al refactor comprobó la frontera de producción: `uv sync` instaló `modelo-citas` 0.2.0 desde S3 y Docker construyó la API sin copiar `model-package/`, sin descargar el dataset y sin reentrenar. También se ejecutaron las pruebas del paquete y la suite completa del proyecto.

Resultado de la validación posterior al refactor: 202 pruebas aprobadas.

La suite completa se ejecutó con el grupo de dependencias de investigación y no produjo fallos funcionales. Se observaron advertencias de deprecación asociadas con NumPy/Joblib, pero no impidieron la ejecución actual.

7. API y tablero

El paquete `modelo-citas` se integró con el backend desarrollado en FastAPI. Al iniciar la aplicación, el backend carga el modelo empaquetado y queda disponible para responder las consultas enviadas desde la API o desde el tablero.

Los endpoints principales son:

Método	Ruta	Función
GET	/api/estado	Estado del modelo
POST	/api/recomendar	Generar recomendaciones
GET	/api/insights	Información analítica
GET	/api/ejemplo	Ejemplo real del corpus
GET	/docs	Documentación Swagger/OpenAPI

Las pruebas realizadas desde Swagger devolvieron código HTTP 200 y resultados reales para el endpoint de recomendación. El endpoint `/api/estado` confirmó que el paquete cargado es `modelo-citas 0.2.0` sobre un catálogo de 19.776 artículos. La ficha técnica expuesta por ese endpoint reporta cinco coeficientes del reordenador, coherentes con la variante `v1 Random` actualmente empaquetada.

El tablero se encuentra en `src/app/`. Desde la interfaz, el usuario puede escribir un contexto, escoger cuántas recomendaciones desea recibir y consultar el ranking generado. Al seleccionar un artículo también puede revisar su resumen.

Además del recomendador, la interfaz presenta tres apartados de análisis: estudio de los datos, desempeño del modelo y diagnóstico de negativos. Así, desde una misma interfaz es posible probar el recomendador y consultar información sobre los datos y el desempeño obtenido durante los experimentos.

8. Uso, instalación y reproducibilidad

8.1 Uso del prototipo

El flujo para utilizar el tablero es el siguiente:

1. abrir la aplicación en el navegador;
2. escribir o pegar un fragmento académico en inglés;
3. escoger el número de recomendaciones;
4. pulsar *Recomendar citas*;
5. revisar los artículos recuperados y sus resúmenes.

También puede utilizarse la opción *Usar un ejemplo real* para cargar una consulta conocida del conjunto de datos y observar el comportamiento del recomendador.

Los resultados deben entenderse como apoyo a la búsqueda bibliográfica. La pertinencia de una referencia debe revisarse antes de utilizarla en un documento académico.

8.2 Instalación

El flujo principal para preparar el proyecto es:

```
git clone https://github.com/Byron971/microproyecto-local-citation.git
cd microproyecto-local-citation
docker compose up --build
```

Con Docker Compose, los puntos de acceso son:

- tablero: <http://localhost:8080>;
- estado de la API: <http://localhost:8000/api/estado>;
- Swagger/OpenAPI: <http://localhost:8000/docs>.

El build instala directamente el wheel `modelo-citas 0.2.0`; no ejecuta DVC ni entrenamiento.

Para reproducir experimentos, datos y pruebas se utiliza el grupo **research**:

```
uv sync --group research
uv run --group research dvc pull
uv run --group research pytest
```

La reproducibilidad del proyecto se apoya en Git y GitHub para el código, DVC para los datos de investigación, MLflow para los experimentos, uv para las dependencias y pytest/Tox para las pruebas. La capa de producción queda desacoplada del entrenamiento mediante el wheel versionado.

9. Estado al cierre de la Entrega 3

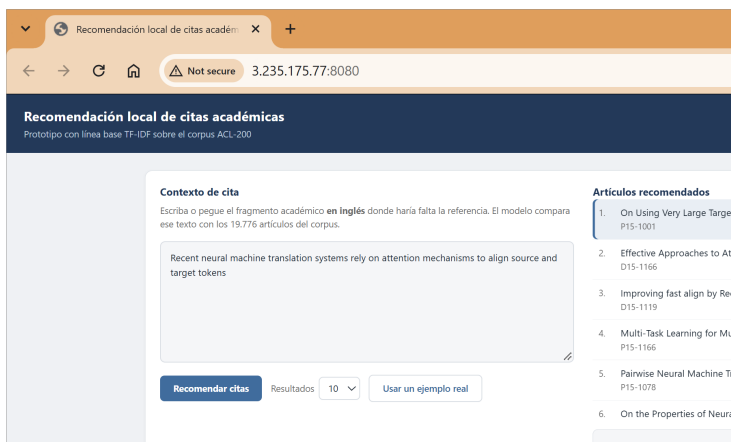
Al cierre de este reporte se encuentran implementados los componentes de datos, experimentación, empaquetamiento, API, tablero y contenedores. El estado se presenta separando explícitamente la mejor variante experimental del modelo que se encuentra empaquetado y demostrado en la API.

Componente	Estado
Repositorio Git y colaboración	Realizado
Datos versionados con DVC	Realizado
Modelos y comparación experimental	Realizado
Seguimiento con MLflow	Realizado
Mejor variante experimental (v2 Random)	Realizado
Modelo empaquetado (v1 Random, wheel 0.2.0)	Realizado
API con modelo empaquetado	Realizado
Tablero integrado	Realizado
Pruebas automatizadas	202 pruebas aprobadas
Manual de usuario e instalación	Actualizado
Dockerización	Realizado
Despliegue mediante contenedores	Realizado
Despliegue en AWS EC2 (post-refactor)	Realizado
Validación post-refactor	Local y en la nube
Video final	Pendiente
Reporte final de aportes	Realizado

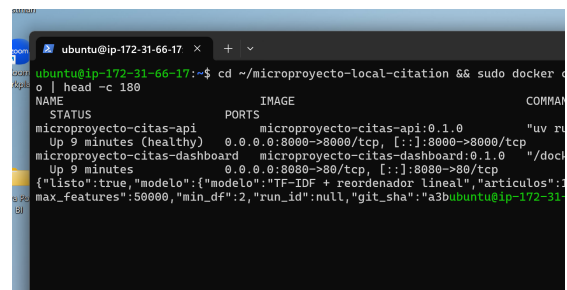
Tabla 2: Estado de los componentes al cierre de la Entrega 3.

Después del refactor de responsabilidades se repitió la validación del sistema. La suite completa finalizó con 202 pruebas aprobadas. En Windows 11, Docker Desktop sobre WSL2 construyó y levantó los servicios con `docker compose up -build`, y el healthcheck de la API respondió correctamente. El endpoint `/api/estado` reportó `listo=true`, paquete `modelo-citas` 0.2.0 y 19.776 artículos. Desde `localhost:8080` se cargó un ejemplo real y el tablero devolvió un ranking de recomendaciones con sus resúmenes. Esta validación confirma la arquitectura de producción posterior al refactor, pero no implica que la variante v2 experimental ya esté publicada en el wheel. El procedimiento actualizado se encuentra en `docs/manual_instalacion.md`.

El mismo sistema se desplegó sobre una instancia EC2 de AWS Academy (`t2.medium`, Ubuntu 24.04) con la arquitectura posterior al refactor. Sobre la instancia se instalaron Docker y Compose, se clonó el repositorio y se levantaron los dos servicios con `docker compose up -d -build`. El `security group` expone únicamente el puerto 8080, por el que Nginx sirve el tablero y enruta las peticiones hacia la API por la red interna de Compose. No se requieren credenciales de AWS en la instancia: el wheel `modelo-citas` 0.2.0 se descarga desde una URL pública de S3.



(a) Tablero servido desde la IP pública de EC2



(b) Contenedores activos en la instancia

Figura 6: Despliegue en la nube posterior al refactor: el tablero responde desde la IP pública y los dos contenedores permanecen activos, con la API en estado *healthy*.

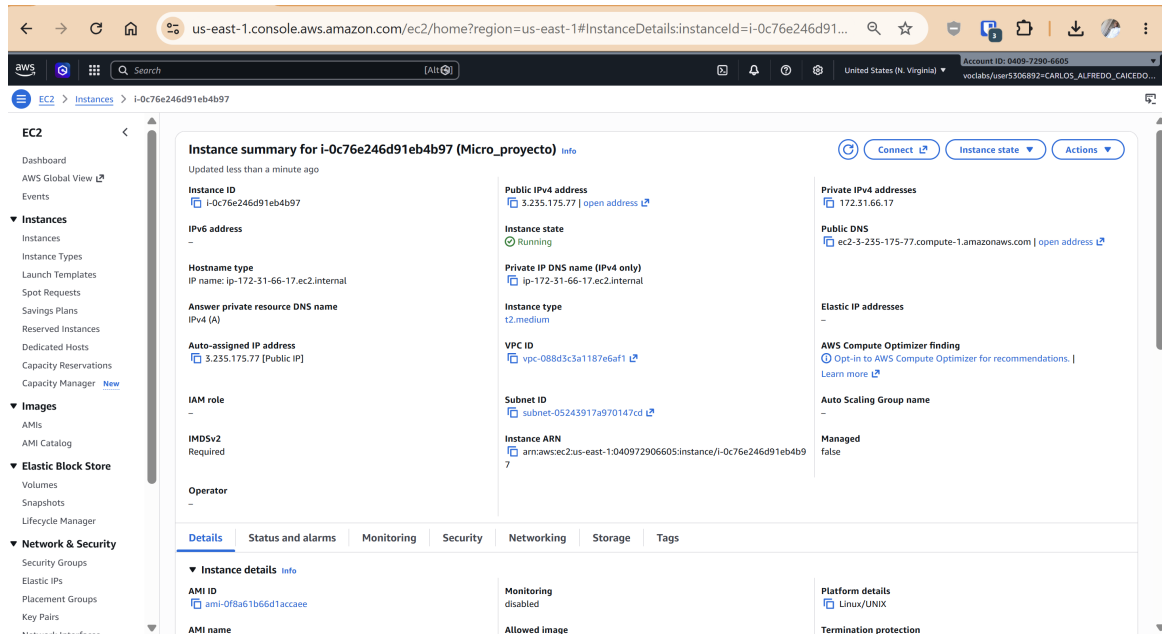


Figura 7: Ficha de la instancia en la consola de EC2. La IP pública 3.235.175.77 coincide con la del tablero y la IP privada 172.31.66.17 con el *prompt* de la sesión SSH, de modo que las tres evidencias corresponden a la misma máquina.

Para cerrar la entrega queda grabar y publicar el video de síntesis e incorporar su enlace. Si el equipo decide que la variante v2 debe ser también la versión servida, será necesario publicar un nuevo wheel con `include_metadata=true` y repetir la validación de Docker. Mientras eso no ocurra, este reporte diferencia explícitamente entre v2 experimental y v1 desplegada.

Soportes principales de la entrega: repositorio GitHub <https://github.com/Byron971/microproyecto-local-citation>; datos en `data/raw.dvc` y `data/processed.dvc`; manuales en `docs/manual_usuario.md` y `docs/manual_instalacion.md`; artefactos de contenedores `Dockerfile`, `Dockerfile.dashboard` y `docker-compose.yml`; evidencias de MLflow y despliegue en `reportes/reporte03/`.

10. Trabajo en equipo

El trabajo se ha organizado mediante ramas, issues, commits y Pull Requests. Los cambios de mayor impacto se revisan antes de incorporarse a `main`. El repositorio también cuenta con CODEOWNERS, lo que permite solicitar revisión a otros integrantes antes de aceptar determinados cambios.

Durante esta fase se revisó la integración final del modelo y el refactor que separó entrenamiento y producción. El código de investigación quedó en `model-package/`, mientras la API instala un wheel versionado desde S3. Después del refactor se ejecutaron 202 pruebas y se validó el flujo Docker → API → modelo → tablero mediante una recomendación real. La revisión también permitió identificar que el wheel 0.2.0 mantiene la configuración v1 de cinco características, mientras v2 sigue siendo la mejor variante experimental.

Integrante	Aportes principales
John Arias	Comparación Random vs. Hard y evidencias de MLflow; automatización de conexión a MLflow en AWS; preparación reproducible de DVC; integración del tablero con la API; dockerización de API y tablero, Docker Compose y documentación del despliegue en AWS; manual de usuario y consolidación del reporte. Para el corte del 20 de septiembre coordinó además la evaluación global, integró el pipeline de métricas, los clientes de modelos comerciales/open-weight y realizó la primera validación manual por parte del equipo sobre 20 casos piloto.
Carlos Duque	Desarrollo de la maqueta inicial en HTML y FastAPI; implementación de una aplicación para servir el tablero y predicciones; empaquetamiento del modelo como <code>modelo-citas</code> ; construcción del paquete instalable y refactor final para separar entrenamiento y producción. Esta última arquitectura publica el modelo como wheel versionado en S3 y permite que FastAPI y Docker lo consuman sin copiar <code>model-package/</code> , sin DVC y sin entrenar durante el build.
José Zambrana	Desarrollo del reordenador supervisado, extracción de características, regresión logística, generación de negativos difíciles, evaluación mediante Recall@K y MRR y documentación de resultados; apoyo en DVC y actualización de evidencias de MLflow. También diseño y versionó los tres prompts de función de cita (zero-shot genérico, zero-shot detallado y few-shot), el conjunto piloto y el análisis de estabilidad y errores de formato.
Carlos Caicedo	Desarrollo y evaluación de la línea base TF-IDF; configuración inicial de MLflow y remotos DVC; diagnóstico de negativos; implementación y evaluación del reordenador versión 2 con características de metadatos; documentación del modelo final; creación y verificación del manual de instalación y auditoría desde entorno limpio, incluida evidencia de Docker y despliegue.

Tabla 3: Resumen de los aportes realizados durante la Entrega 3.

La tabla resume contribuciones verificables en el historial del repositorio, issues y Pull Requests. De acuerdo con la guía del curso, la contribución individual se respalda mediante commits, el reporte de trabajo en equipo y la sustentación; el video puede apoyarse en esta distribución para presentar el trabajo realizado.

Como extensión adicional se versionaron tres estrategias de *prompting*, el parser común de nueve puntajes y el pipeline de métricas con Precision, Recall, F1 Macro/Micro y matrices de confusión. Se completó una primera validación manual sobre 20 casos piloto y se ejecutaron corridas provisionales con Qwen3:8B, Gemini y Cohere; OpenAI quedó implementado, pero sin predicciones por falta de saldo disponible. El detalle se conserva en `docs/evaluacion_comparativa_provisional_20sep.md`.

Posteriormente se completó la segunda validación humana independiente sobre los mismos 20 casos, realizada sin consultar la primera hoja ni las sugerencias automáticas. El acuerdo fue de 12 coincidencias y 8 desacuerdos, es decir una tasa de 0,60 y un *Cohen Kappa* de 0,4839, valor que corresponde a un acuerdo moderado y se ubica dentro del rango habitual para taxonomías de función de cita con nueve clases. Tras reconciliar los desacuerdos quedó congelado el *Test Gold* definitivo en `annotations/citation_function/test_gold.jsonl`, que difiere del conjunto provisional en un único caso.

Tres limitaciones acompañan esa cifra. Primero, el Kappa compara una anotación asistida con explicaciones contra una independiente, no dos anotaciones ciegas. Segundo, la reconciliación de los ocho desacuerdos también fue asistida, y siete se resolvieron hacia la etiqueta del primer anotador. Tercero, el conjunto definitivo cubre solo 5 de las 9 categorías —Background, Application, Evidence, Improvement y Further Reading—, por lo que el F1 Macro sobre las nueve clases permanece penalizado por las cuatro ausentes; ampliar el piloto más allá de 20 casos es la vía para levantar esa restricción. Las corridas de los modelos sobre este conjunto definitivo quedan pendientes.

11. Conclusiones

La comparación experimental identificó a la versión 2 con negativos aleatorios como la mejor variante medida: alcanzó Recall@10 de 0,3793 y MRR@10 de 0,2166, mientras que la versión 2 con negativos difíciles obtuvo 0,2288 y 0,0995. Sin embargo, la versión actualmente empaquetada y demostrada en la API es v1 **Random**, incluida en `modelo-citas` 0.2.0. Esta distinción evita atribuir al sistema desplegado métricas que corresponden a una variante todavía no publicada en el wheel.

El experimento también permitió identificar una limitación importante. Con Recall@100 de 0,5160, cerca del 48,4 % de las consultas de validación no tiene la cita correcta dentro de los cien candidatos iniciales. Por ello, además de mejorar el reordenador, una línea prioritaria de trabajo es fortalecer la etapa de recuperación.

Además de mejorar el modelo, durante esta entrega se avanzó en la construcción de una solución que pueda repro-

ducirse y reutilizarse fuera del entorno de experimentación. Los datos están versionados con DVC, los experimentos quedan registrados en MLflow, el modelo puede instalarse como wheel y ya es consumido por FastAPI y por el tablero.

La contenerización quedó completada y verificada después del refactor. La imagen de la API instala el wheel `modelo-citas` 0.2.0 publicado en S3 y no incorpora dependencias de entrenamiento ni necesita descargar los datos durante el build. La arquitectura final fue validada localmente con Docker Compose y desplegada sobre una instancia EC2 de AWS Academy, con API saludable y una recomendación real devuelta desde el tablero servido por la IP pública.

Para cerrar la Entrega 3 queda pendiente el video de síntesis y su enlace final. Si se desea desplegar también la mejor variante experimental, el siguiente paso es publicar un nuevo wheel con v2 y repetir la prueba contenerizada. En la extensión de clasificación de función de cita ya se completaron la segunda validación humana independiente, el cálculo de acuerdo y la reconciliación, con un Cohen Kappa de 0,4839; resta repetir la evaluación de los modelos sobre el Test Gold definitivo antes de presentar esos resultados como definitivos. En el recomendador principal, el Recall@100 de 0,5160 muestra que un margen importante de mejora está en la recuperación de candidatos, por ejemplo mediante representaciones semánticas más ricas.

Referencias

- Gu, N. (2022). *Local-Citation-Recommendation* [Computer software]. GitHub. <https://github.com/nianlonggu/Local-Citation-Recommendation>
- Gu, N., Gao, Y., & Hahnloser, R. H. R. (2022). Local citation recommendation with hierarchical-attention text encoder and SciBERT-based reranking. *Advances in Information Retrieval*, 274–288. Springer. https://doi.org/10.1007/978-3-030-99736-6_19
- Treveil, M., & Dataiku Team. (2021). *Introducing MLOps: How to scale machine learning in the enterprise*. O'Reilly Media.